STUDENT PERFORMANCE ANALYTICS SYSTEM

Python with AI – Mini Project Documentation

Submitted by: B. Malleeshwari

Department: CSE(AI&ML)

College: Anurag Engineering College

1. Abstract

The Student Performance Analytics System is a Python-based data analysis project that processes student marks and attendance-related records to produce useful academic summaries. It uses Pandas to organize student data in a DataFrame and NumPy for numerical calculations. The system calculates total and average marks, assigns grades, identifies pass/fail results, summarizes subject-wise performance, and lists the top-performing students.

2. Aim

To develop a Python-based system that analyzes student academic performance and presents clear, meaningful results from a student dataset.

3. Objectives

- Create and load a dataset containing student details and marks.
- Use Pandas DataFrames to organize and process records.
- Use NumPy for numerical calculations.
- Apply functions, conditions, and loops to calculate grades and results.
- Calculate total marks, average marks, and class statistics.
- Analyze subject-wise averages and identify the top five students.
- Save the analyzed results to a CSV report.

4. Technologies Used

- Python
- Pandas
- NumPy
- Google Colab
- CSV (Comma-Separated Values) dataset

5. Dataset Description

The sample dataset contains 20 student records. Its columns are Student_ID, Name, Department, Python, Mathematics, AI, and Attendance. The three subject columns are used to calculate each student's total and average marks. The dataset is stored in students.csv.

6. Methodology / Algorithm

1. Import Pandas and NumPy.

2. Create the student dataset and save it as students.csv.

3. Read students.csv into a Pandas DataFrame.

4. Select Python, Mathematics, and AI as the subject columns.

5. Calculate total marks and average marks for every student.

6. Assign a grade using the defined average-mark ranges.

7. Mark a student as Pass when every listed subject mark is at least 35; otherwise mark Fail.

8. Calculate class statistics and subject-wise averages.

9. Display the top five students by average marks.

10. Save the complete analyzed DataFrame as student_performance_report.csv.

7. Grading Rules

A+ : Average ≥ 90

A : Average ≥ 80 and < 90

B : Average ≥ 70 and < 80

C : Average ≥ 60 and < 70

D : Average ≥ 50 and < 60

F : Average < 50

Pass/Fail rule: a student passes if each of the three subject marks (Python, Mathematics, and AI) is at least 35; otherwise, the result is Fail.

8. Key Features

- Student dataset management using Pandas.
- Total and average marks calculation.
- Automatic grade assignment.
- Pass/fail classification.
- Highest, lowest, and class average calculations.
- Subject-wise average analysis.
- Top-five student identification.
- Export of the final report to CSV.

9. Implementation

The program imports Pandas and NumPy and reads the CSV dataset using pd.read_csv(). It calculates totals with DataFrame.sum(axis=1) and averages with DataFrame.mean(axis=1). A user-defined function assigns grades based on the average. Another function checks whether each subject mark meets the passing threshold. The program then prints class statistics, calculates subject-wise averages with NumPy, displays the five highest averages, and exports the processed DataFrame to a CSV file.

10. Expected Output

The program displays a student performance table with total marks, average marks, grade, and result. It also displays the total number of students, class average, highest and lowest averages, pass/fail counts, subject-wise averages, and the top five students. The final processed data is saved as student_performance_report.csv.

Note: Insert your own screenshots from the successfully executed Google Colab notebook in this section before final submission.

11. Result

The Student Performance Analytics System was executed successfully in Google Colab. The program generated subject-wise averages and displayed the top five students based on average marks. It also produced a processed CSV report containing calculated performance fields.

12. Conclusion

This project demonstrates how Python, Pandas, and NumPy can be used to organize and analyze student marks. It automates common calculations and presents results in a readable format, helping users review student and class performance efficiently.

13. Challenges and Learning

The project provides practice in creating a dataset, loading CSV data, working with DataFrames, writing reusable functions, applying conditions, using loops, performing numerical analysis, and exporting results. Before submission, verify that the code and CSV files run correctly in a fresh notebook session.

14. GitHub Repository

Repository URL: Add your public GitHub repository URL here after creating and uploading the project files.

Recommended repository files:

- main.py
- students.csv
- student_performance_report.csv

• README.md

• Student_Performance_Analytics_Documentation.docx

• Student_Performance_Analytics_Documentation.pdf

Appendix A – Main Program Code

```
import pandas as pd

import numpy as np


df = pd.read_csv("students.csv")

subjects = ["Python", "Mathematics", "AI"]


df["Total"] = df[subjects].sum(axis=1)

df["Average"] = df[subjects].mean(axis=1).round(2)


def calculate_grade(avg):

    if avg >= 90:

        return "A+"

    elif avg >= 80:

        return "A"

    elif avg >= 70:

        return "B"

    elif avg >= 60:

        return "C"

    elif avg >= 50:

        return "D"

    return "F"


def calculate_result(row):

    if all(row[subject] >= 35 for subject in subjects):

        return "Pass"

    return "Fail"
```

```
df["Grade"] = df["Average"].apply(calculate_grade)

df["Result"] = df.apply(calculate_result, axis=1)


print("STUDENT PERFORMANCE REPORT")

print(df.to_string(index=False))


print("\nCLASS STATISTICS")

print("Total Students:", len(df))

print("Class Average:", round(df["Average"].mean(), 2))

print("Highest Average:", df["Average"].max())

print("Lowest Average:", df["Average"].min())

print("Students Passed:", (df["Result"] == "Pass").sum())

print("Students Failed:", (df["Result"] == "Fail").sum())


print("\nSUBJECT-WISE AVERAGES")

for subject in subjects:

    print(subject, "Average:", round(np.mean(df[subject]), 2))


print("\nTOP 5 STUDENTS")

print(df.nlargest(5, "Average")[["Student_ID", "Name", "Average", "Grade"]].to_string(index=False))


df.to_csv("student_performance_report.csv", index=False)

print("\nProject executed successfully!")
```